

OE-EC604B — Operating System

Semester VI • 3L:0T:0P • 3 Credits

Module 8 Exam Notes: File Systems

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	File and Directory Concepts	2
1.1	File Concepts & Access Mechanisms	2
1.2	Directory Structures	2
2	File System Implementation and Allocation Methods	2
2.1	1. Contiguous Allocation	3
2.2	2. Linked Allocation	3
2.3	3. Indexed Allocation	3
3	Free Space Management	3

1. File and Directory Concepts

The file system provides the online storage and access mechanism for both data and programs of the operating system.

1.1 File Concepts & Access Mechanisms

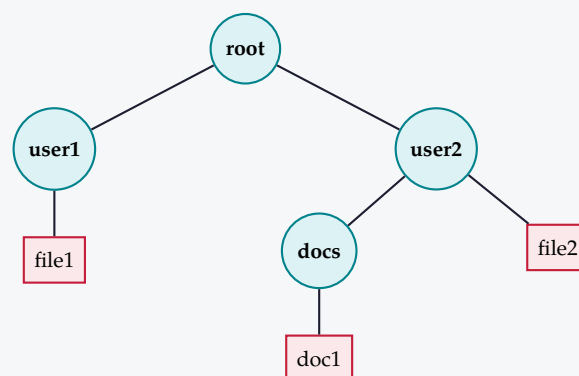
- **File Concept:** A file is a named collection of related information that is recorded on secondary storage.
- **Access Methods:**
 - **Sequential Access:** Information is processed in order, one record after another (e.g., tape storage).
 - **Direct (Random) Access:** A file is viewed as a numbered sequence of blocks. The program can read/write any block instantly in any order (e.g., database queries on disk).
 - **Indexed Access:** An index table is built containing pointers to various blocks. The program searches the index first to retrieve records.

1.2 Directory Structures

A directory is a container that organizes files. Common topologies:

- **Single-Level Directory:** All files are in a single directory. *Problems:* Naming collisions, organization issues.
- **Two-Level Directory:** Each user gets a private User File Directory (UFD) nested under a Master File Directory (MFD).
- **Tree-Structured Directory:** Users can create subdirectories and organize files hierarchically. The most common structure.

Tree-Structured Directory Tree



2. File System Implementation and Allocation Methods

To store files on disk, the OS must map logical file blocks to physical disk sectors. There are three traditional allocation methods:

2.1 1. Contiguous Allocation

- **Working:** Each file occupies a contiguous set of blocks on the disk.
- **Advantages:** Fast sequential read/write operations because the disk head does not need to move. Easy direct access.
- **Disadvantages:** Prone to **external fragmentation**. Hard to grow a file dynamically if adjacent blocks are already allocated.

2.2 2. Linked Allocation

- **Working:** Each file is a linked list of disk blocks. Directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block.
- **Advantages:** No external fragmentation. Files can grow indefinitely.
- **Disadvantages:** Slow random (direct) access (must traverse links sequentially from block 1). Pointers consume storage space. If a pointer gets corrupted, the rest of the file is lost.
- **FAT (File Allocation Table):** A variation of linked allocation. Pointers are stored in a centralized table at the beginning of the volume, caching entries for faster lookup.

2.3 3. Indexed Allocation

- **Working:** Each file has its own **index block**, which contains an array of pointers to all its physical data blocks.
- **Advantages:** Supports direct access without traversing lists. No external fragmentation.
- **Disadvantages:** Index block overhead — wastes space for very small files. If a file grows too large, a single index block is insufficient. (Resolved using linked index blocks, multilevel indexing, or UNIX indirect blocks).

3. Free Space Management

To allocate blocks, the OS tracks all free disk blocks using one of two common methods:

1. **Bit Vector (Bit Map):** An array of bits where each bit corresponds to a disk block (0 = allocated, 1 = free). *Advantages:* Fast, easy to find consecutive blocks. *Disadvantages:* Must be kept in RAM, consumes space for large disks.
2. **Linked List:** All free blocks are linked together. Directory keeps a pointer to the first free block. *Advantages:* Minimal space overhead. *Disadvantages:* Slow to traverse; hard to find contiguous free blocks.

Quick Revision — Module 8

File Systems | OE-EC604B

File Allocation Calculations

Let block size = 1 KB. Pointer size = 4 Bytes.

- **Single Index Block Limit:** An index block can hold $1 \text{ KB} / 4 \text{ B} = 256$ pointers. Max file size = $256 \times 1 \text{ KB} = 256 \text{ KB}$.
- **Two-Level Indexing Limit:** Primary index block points to 256 secondary index blocks. Max file size = $256 \times 256 \times 1 \text{ KB} = 64 \text{ MB}$.

One-Line Definitions

- **File:** A named logical collection of related records stored on secondary storage.
- **Sequential Access:** Access method where data is read in sequence from beginning to end.
- **Direct Access:** Random block access where records are accessed directly by block index.
- **Contiguous Allocation:** Allocation method where file blocks are saved in consecutive disk sectors.
- **Linked Allocation:** Allocation method where blocks are linked via pointers in a list structure.
- **FAT:** File Allocation Table; centralized directory indexing system caching linked pointers.
- **Indexed Allocation:** Allocation method where all block pointers are collected in an index block.
- **Bit Vector:** Binary sequence where each bit represents the free/allocated state of a disk block.

Frequently Asked / Viva Questions

Q1. Compare Contiguous, Linked, and Indexed file allocation methods.

Contiguous is fast, supports direct access, but suffers from external fragmentation and file-growth limitations. Linked eliminates fragmentation and allows files to grow easily but has slow random access and pointer overhead. Indexed supports direct access and file growth with no fragmentation, but has index block space overhead.

Q2. Explain the FAT (File Allocation Table) file system structure.

FAT is a variation of linked allocation. It extracts all block pointers from the data blocks and stores them in a centralized table at the beginning of the partition. The directory entry contains the starting block number, and the table is indexed

Frequently Asked / Viva Questions (contd.)

by block numbers to find subsequent blocks.

Q3. What is an index block in file system implementation? What happens if a file exceeds its capacity?

An index block contains an array of disk block addresses allocated to a file. If a file exceeds the index block capacity, the OS uses: 1. Linked scheme (linking multiple index blocks). 2. Multilevel indexing (an index block containing pointers to other index blocks). 3. Combined scheme (like UNIX inodes).

Q4. Describe free space management using a Bit Vector. Calculate the size of a bit map for a 200 GB disk with 4 KB blocks.

A bit vector uses 1 bit for each disk block: 1 means free, 0 means allocated. *Total Blocks* = $200 \text{ GB} / 4 \text{ KB} = (200 \times 10^9) / (4 \times 10^3) = 50 \times 10^6$ blocks.

Map Size = 50×10^6 bits ≈ 6.25 MB of storage.

Q5. Explain the difference between sequential and direct file access mechanisms.

Sequential access processes records one after another from the beginning, moving a read pointer sequentially. Direct access allows reading/writing blocks in any arbitrary order by specifying the relative block number directly.

Q6. What are the advantages of a tree-structured directory over a single-level directory?

A tree-structured directory allows users to create nested subdirectories to organize files logically, preventing naming conflicts and enabling file organization, sharing, and access control.

Q7. Explain the implementation issues of contiguous allocation.

Finding space for a new file requires executing dynamic allocation algorithms (first-fit, best-fit), which leads to external fragmentation. Also, if a file grows, it may collide with an adjacent allocated file, requiring compaction.

Q8. Why does linked allocation suffer from reliability issues?

Because the blocks are scattered across the disk, and each block contains a pointer to the next. If a single sector gets corrupted, the pointer is lost, rendering all subsequent blocks inaccessible.

Q9. What is the purpose of directory mounting?

Mounting allows the OS to attach multiple physical file systems on different storage devices (hard disks, USB drives) into a single, unified logical directory tree.

Q10. How does the OS optimize directory lookup speed?

By using hash tables or B-trees instead of linear lists to store directory entries, reducing lookup times from $O(N)$ to $O(1)$ or $O(\log N)$.

File Allocation Methods Comparison

Feature	Contiguous	Linked	Indexed
Direct Access	Excellent	Very Poor (linear traversal)	Excellent
External Frag.	Yes	None	None
Storage Efficiency	High	Moderate (pointer overhead)	Low for small files
File Growth	Hard (contiguous block)	Easy	Easy